

Atlas / Argus

Current Implementation Status

What has already been built, what is partially done, and what remains to ship

Prepared from the uploaded backend ZIP, frontend ZIP, conflict-candidates patch ZIP, and blueprint documents.

Scope	Status
Snapshot date	Based on uploaded artifacts present in this ChatGPT workspace on 2026-05-30.
Purpose	Give an AI or engineer a concise, factual view of what currently exists before planning further work.
Verification note	Static inspection was performed. compileall passed in this container; lint/type/test tools were not fully available here.

Contents

1. Executive status snapshot
2. Repository and artifact inventory
3. Backend implementation completed
4. API surface currently present
5. Database, migrations, and persistence completed
6. Security, tenancy, and operations completed
7. Frontend implementation completed
8. Separate conflict-candidates patch completed
9. Verification status
10. Known gaps and unfinished work
11. Recommended next build sequence
12. AI handoff guidance

1. Executive status snapshot

Atlas / Argus is no longer just an idea or early prototype. The uploaded backend is a mature Python 3.12 FastAPI system with a claims-based evidence model, conflict lifecycle, projections, provenance, ingestion idempotency, public/event APIs, search, maps, tenancy, causality, CMS/editorial, audit, metering, and operational deployment files. The uploaded frontend is a real React/Vite/TypeScript/Tailwind scaffold with the main authenticated routes already in place, but it is not yet a sellable production UI.

Area	Current state	Status
Backend core	Claims, sources, events, conflicts, projections, provenance, ingestion runs, outbox, and NTSB importer exist.	Done
Backend product layers	Public events, search, maps, Chronos, Orion, Hermes, Argus, causality, tenancy, CMS/editorial, audit, and metering are present as bounded contexts with routes/tests.	Substantial
Database	49 Alembic migration files through 049_fk_covering_indexes and 61 ORM model classes are present.	Done/advanced
Security	API-key auth with HMAC hashing, analyst/reviewer/admin roles, tenant bindings, production settings validation, rate/body limits, CORS/HSTS/TrustedHost controls.	Done for API-key backend
Frontend app	apps/web exists and has login, case list, dashboard, claims, conflicts, timeline, provenance, reports, audit, layout, API client, and typed DTOs.	Scaffold/partial
BFF/session auth	BFF skeleton exists in atlas-bff-docs-reports.zip; needs production DB, sessions/CSRF, API-key bridge, and apps/web wiring.	Skeleton/partial
Report export	Reports route/UI exists; production immutable generation, storage, version history, and download	Partial

Document upload UI	Documents.tsx route exists; needs BFF/storage/parser wiring.	Partial
Marketing site	No apps/marketing site is present in the uploaded frontend ZIP.	Not done
Conflict candidates fix	Applied in atlas-argus-docs-reports.zip; patched conflicts.py and schemas match 2026-05-30 patch dates.	Applied

2. Repository and artifact inventory

2.1 Uploaded artifacts inspected

Artifact	What it contains	Current use in this report
atlas-argus-production-ready(9).zip	Main backend repository: Python package atlas, tests, Alembic migrations, Docker/deploy/ops files, release docs.	Primary source for backend status.
atlas-web-app(3).zip	React/Vite/TypeScript/Tailwind authenticated app scaffold under apps/web.	Primary source for frontend status.
atlas-conflict-candidates-patch.zip	Targeted backend and frontend patch for stable conflict claim-candidate selection.	Recorded as ready-to-apply patch, not base-state.
atlas-argus-blueprint-updated(2).pdf	Product and frontend roadmap update explaining the app scaffold and remaining product gaps.	Used to align current-state interpretation.
atlas-argus-implementation-plan-updated(2).pdf	Implementation plan update describing backend package paths, roles, BFF need, conflict queue issues, and route expectations.	Used to identify intended next work.
Aviation-Safety-Atlas-Implementation-Blueprint(2).pdf	Short architecture blueprint emphasizing BFF, unified evidence panel, document ingestion, and report export.	Used for cross-checking open gaps.

2.2 Static inventory from the main backend ZIP

Metric	Value
Python package root	src/atlas
Backend source files	202
Test files	133
Alembic migration files	49
Current documented migration head	049_fk_covering_indexes
ORM model classes excluding Base	61
FastAPI router modules	19
Route decorators found by static scan	109
Frontend app files inspected	39
Frontend src files inspected	28

2.3 Real import and route conventions

- **Python import root:** atlas, not atlas_argus.

- **API prefix:** routers are mounted under /api/v1, with admin mounted under /api/v1/admin.
- **Role names:** analyst, reviewer, admin. Use these names in UI/BFF work.
- **Authentication model:** the backend currently authenticates with X-API-Key, not human login sessions.
- **Frontend API client:** apps/web calls a base URL via VITE_API_BASE_URL and sends cookies with credentials included.

3. Backend implementation completed

3.1 Clean architecture structure

- domain/: pure domain entities, enums, exceptions, services, interfaces, and bounded contexts.
- application/: use cases, DTOs, unit-of-work protocol, ingestion pipeline helpers, and application services.
- infrastructure/: SQLAlchemy ORM, repositories/session/unit-of-work, ingestion readers, outbox worker, and metrics.
- presentation/: FastAPI app, routers, schemas, dependencies, middleware, responses, metrics, and Typer CLI.
- tests/: API, domain, application, infrastructure, and integration coverage, mirrored around the source tree.

3.2 Evidence-chain core already built

Capability	What currently exists
Source ingestion	Source, IngestionRun, RawSnapshot, idempotency keys, source_record_id handling, source field mappings, raw payload preservation, and normalization.
Claim model	Claim lifecycle supports RAW, CONFIRMED, MANUAL_OVERRIDE, and SUPERSEDED. ClaimHistory records creation, updates, supersession, reactivation, and curator actions.
Conflict lifecycle	ClaimConflict supports OPEN and RESOLVED, optimistic version checks, resolve/reopen workflows, activity logs, winning_claim_id, and manual overrides through CuratorOverride source.
Projection builder	ProjectedAccidentRecord derives the current public/projection view from active claims and conflict winner policy, including DISPUTED markers for unresolved conflicts.
Provenance	APIs and use cases expose evidence chains, claims, source snapshots, conflict history, and projection lineage.
Outbox	OutboxEvent and outbox worker decouple claim updates from downstream processing and support retry/dead-letter lifecycle.
Duplicate/event resolution	Event identity index, registration normalization, duplicate review, merge use case, and canonical event resolution are present.
Audit explanation	Field, claim, and source verification explanation endpoints exist for non-technical inspection of why a value appears.

3.3 Domain/product subsystems already present

Subsystem	Current implementation state
Atlas core	Claims, events, projections, conflicts, provenance, ingestion, winner policy, and completeness services.
Hermes	Source registry, crawl targets, fetch queue/jobs, fetched documents, source changes, SSRF/security-focused fetch behavior, and worker CLI.

Orion	Entity registry, identifiers, relationships, entity-claim links, extraction, search, relationship APIs, and review status.
Chronos	Timeline event extraction, event links, sequence reviews, timeline APIs, and timestamp precision models.
Argus	Signal detection, severity/review services, signal evidence, signal review, listing, and review APIs.
Public events/publication	PublicEventPage, revisions, slug handling, publication workflow, public list/detail/evidence/timeline/related/audit APIs.
Search	Search index entry model, reindex use case, structured search events endpoint, admin reindex endpoint.
Maps	MapIndexEntry, map indexing use case, events endpoint, cluster endpoint, PostGIS-oriented map models.
Causality	HFACS taxonomy, SHELO factors/interactions, event attributions, editorial attach/update/delete APIs, public reads.
Tenancy/enterprise	Tenant, membership, tenant sources, ingestion runs, tenant claims, event overlays, safety reports, cross-reference results, tenant evidence endpoints.
CMS/editorial	Glossary, methodology pages, changelog entries, revisions, and public/editorial CMS APIs.
NL search	Deterministic NL search query parsing/use cases, saved queries, query logs, and endpoints.
Metering	Usage events, daily rollups, tenant usage, admin usage summary, and rollup endpoint.

3.4 NTSB eADMS importer already built

- Pure mapper in application/ingestion/sources/ntsb_eadms.py converts eADMS rows into normal Atlas ingestion submissions.
- Infrastructure reader exports/reads avall.mdb-derived CSV data and joins records.
- Typer CLI supports export, dry-run, and load workflows.
- Importer uses source_record_id = ev_id and content-addressed idempotency keys so re-runs are deterministic and resumable.
- Documentation states it was verified against the full public avall.mdb dataset: 30,516 records mapped, 0 duplicate-field-name submissions, and deterministic idempotency keys.

4. API surface currently present

The inspected backend wires the following router modules under /api/v1 unless noted. This is a high-level static route inventory, not a complete OpenAPI extract.

Router module	Routes found	Examples
accidents	2	GET /{event_id}, POST /{event_id}/reproject
admin	10	POST /projections/rebuild, GET /projections/verify, GET /outbox, POST /outbox/process, GET /metrics...
argus	5	POST /run, GET /signals, GET /signals/page, GET /signals/{signal_id}, POST /signals/{signal_id}/review
audit	3	GET /events/{event_id}/fields/{field_name}/explanation, GET /claims/{claim_id}/explanation, GET /sources/{snapshot_id}/verification
causality	11	GET /public/hfacs/taxonomy, GET /public/events/{slug}/hfacs, GET /public/events/{slug}/shelo, POST

		/editorial/events/{event_id}/hfacs, PUT /editorial/events/{event_id}/hfacs/{attribution_id}...
chronos	3	POST /events/{event_id}/extract, GET /events/{event_id}/timeline, GET /reviews/pending
cms	12	GET /public/glossary, GET /public/glossary/{term}, GET /public/methodology, GET /public/methodology/{slug}, GET /public/changelog...
conflicts	5	GET , GET /{conflict_id}, GET /{conflict_id}/history, POST /{conflict_id}/resolve, POST /{conflict_id}/reopen
editorial	13	POST , GET , GET /{page_id}, PATCH /{page_id}, POST /{page_id}/submit...
hermes	10	POST /sources, GET /sources, POST /targets, GET /targets, POST /targets/{target_id}/enqueue...
ingestion	1	POST /sources/{source_id}
maps	2	GET /events, GET /events/cluster
metering	3	GET /usage, GET /summary, POST /rollups
nl_search	4	POST , POST /saved, GET /saved, DELETE /saved/{saved_id}
orion	6	POST /events/{event_id}/extract, GET /events/{event_id}/entities, GET /entities/search, GET /entities/{entity_id}, GET /entities/{entity_id}/relationships...
provenance	1	GET /{event_id}/provenance
public	6	GET , GET /{slug}, GET /{slug}/evidence, GET /{slug}/timeline, GET /{slug}/related...
search	1	GET /events
tenancy	11	POST /sources, GET /events/{event_id}/overlay, PUT /events/{event_id}/overlay, GET /events, POST /ingestions...

4.1 Important route implications for future frontend work

- The current frontend calls /api/v1/auth/login, /api/v1/auth/me, and /api/v1/auth/logout, but those auth endpoints are not present in the backend router inventory. They belong in the BFF/session layer.
- The base conflict route exposes list/detail/history/resolve/reopen, but the stable conflict candidate endpoint from the patch ZIP is not present in the production-ready tree.
- Public routes still require X-API-Key in this backend. Public means published/read-only Atlas record, not anonymous internet access.
- Admin routes exist for projection rebuild, verification, outbox, metrics, event merge, duplicate review, source field mapping, and search reindexing.

5. Database, migrations, and persistence completed

5.1 ORM model families present

Family	Representative models
Core evidence	SourceModel, IngestionRunModel, RawSnapshotModel, AccidentEventModel, ClaimModel, ClaimHistoryModel, ClaimConflictModel, ConflictActivityLogModel, ProjectedAccidentRecordModel
Operations	OutboxEventModel, OutboxWorkerHeartbeatModel, AccidentProjectionHistoryModel, ArchiveManifestModel, ApiKeyModel
Identity/review	PendingDuplicateReviewModel, EventIdentityIndexModel
Orion/Chronos/Hermes/Argus	OrionEntityModel, OrionRelationshipModel,

	ChronosTimelineEventModel, HermesSourceModel, HermesFetchJobModel, ArgusSignalModel, and related evidence/review/link models
Public/search/maps	PublicEventPageModel, PublicEventPageRevisionModel, SearchIndexEntryModel, MapIndexEntryModel
Tenancy/enterprise	TenantModel, TenantMembershipModel, TenantSourceModel, TenantIngestionRunModel, TenantClaimModel, TenantEventOverlayModel, TenantSafetyReportModel, TenantEventAssociationModel, TenantCrossrefResultModel
CMS/causality/NL/metering	GlossaryTermModel, MethodologyPageModel, ChangelogEntryModel, HFACS/SHELO models, NIQueryLogModel, SavedNIQueryModel, UsageEventModel, UsageDailyRollupModel

5.2 Migration coverage

- The repository contains 49 migration files from 001_initial_schema.py through 049_fk_covering_indexes.py.
- The migrations cover initial schema, source reliability constraints, identity indexes, claim/conflict hot paths, Orion, Chronos, Hermes, Argus, public pages, editorial workflow, search index, tenants and overlays, maps, CMS, tenant ingestion, causality, NL search, metering, tenant RLS, cross-reference results, echo enums/outbox, and FK covering indexes.
- The release notes state the Alembic graph had a single linear head and that migration round-trip was verified in the release environment.

6. Security, tenancy, and operations completed

Area	What is already in place
API auth	X-API-Key based dependency, HMAC-SHA256 hashing support, previous-secret rotation bridge, in-process LRU cache, and ApiKeyModel with user_id, role, tenant_id, tenant_role.
Roles	Canonical Role enum: analyst, reviewer, admin. Role-based dependencies gate read/review/admin actions.
Tenant isolation	Tenant models, tenant-scoped data models, tenant APIs, tenant DB URLs, RLS documentation, tenant context helpers, and RLS-oriented tests.
Request hardening	TrustedHost, CORS controls, security headers/HSTS, request body size limit, and in-memory rate limit middleware.
Runtime validation	Settings validate production requirements including DB URL split, API key hash secret, allowed hosts, CORS, metrics controls, and related deployment safety.
Deployment	Dockerfile, docker-compose, deploy/free stack with Caddy/Postgres/Redis/PgBouncer/API/worker, Gunicorn/Uvicorn config, start script.
Operations	Operations guide, production cutover checklist, API key rotation runbook, Prometheus alert rules, k6/load test artifacts, readiness/health endpoints.
Release verification docs	RELEASE.md records compile, lint, format, strict mypy, unit/API/integration tests, migration checks, secret scans, Docker build, and vulnerability scans as passing in the release environment.

7. Frontend implementation completed

7.1 apps/web exists and should be preserved

- The frontend is not absent. The uploaded ZIP contains apps/web, a React/Vite/TypeScript/Tailwind application.
- It uses TanStack Query, React Router, Lucide icons, and a centralized API wrapper.
- The existing scaffold should be hardened and wired through a BFF rather than rebuilt from scratch.

7.2 Current frontend route surface

Route/file	Current purpose
/login	Login screen using AuthProvider and /api/v1/auth/login expectations.
/app and /app/cases	Case list and app landing flow backed by public events query.
/app/cases/:slug	Case dashboard using case detail, conflict list, and accident projection hooks.
/app/cases/:slug/claims	Claims view with left claims list, center detail, and right evidence panel.
/app/cases/:slug/conflicts	Conflict queue route with filters, detail panel, resolve/reopen forms, history, evidence panel, and optimistic 409 concepts.
/app/cases/:slug/timeline	Chronos-style timeline view with selectable entries and detail panel.
/app/cases/:slug/provenance	Provenance exploration with claims, conflicts, projection history, and evidence chain panel.
/app/cases/:slug/reports	Reports route exists, but is currently a placeholder/locked surface rather than a production PDF generation workflow.
/app/audit	Read-only audit view for activity and page audit data.

7.3 Current frontend technical pieces

Piece	What exists
App shell	AppLayout, Sidebar, TopBar, CaseWorkspaceLayout, FullWidthLayout, protected-route behavior via AuthProvider.
API client	src/lib/api.ts centralizes fetch calls, VITE_API_BASE_URL, credentials include, camel/snake key conversion, API error handling, and 409 detection helper.
Auth provider	AuthProvider uses TanStack Query to load /auth/me, login, logout, role checks, and permission checks.
Case feature API	Hooks for public event list/detail/evidence/timeline/audit.
Conflict feature API	Hooks for list/detail/history/resolve/reopen and projection fetch; optimistic invalidation after mutation.
UI components	StatusChip, ReliabilityChip, ConfidenceChip, EmptyState, skeletons, ErrorPanel, ConflictModifiedBanner, Sidebar case switcher, TopBar search/logout.
Typed DTOs	src/types/api.ts mirrors backend response structures for current frontend routes.

7.4 Frontend is still a workbench, not a finished product

- It expects auth/session endpoints that are not present in the FastAPI backend. The missing layer is the BFF.
- The reports page is not a completed preview/generate/download/export history workflow.
- There is no production document upload, parse-status, extracted-text, or ingestion trigger UI.
- There is no separate public marketing website under apps/marketing.
- Global search is not fully activated as a production feature in the app shell.
- Playwright/integration tests are not present in the uploaded apps/web scaffold.

8. Separate conflict-candidates patch completed

A separate ZIP named atlas-conflict-candidates-patch.zip contains a targeted correctness fix for the conflict resolve UI. It should be treated as a ready patch, not as part of the inspected production-ready base trees unless it has been applied manually in the real repository.

Patch area	What the patch adds/fixes
Backend schema/router	Adds ConflictClaimCandidateDTO and ConflictCandidatesResponse plus GET /api/v1/conflicts/{conflict_id}/candidates.
Backend behavior	Returns stable claim_id, field, value, source metadata, reliability tier, claim type, created_at, is_winning, is_superseded, and superseded_by_claim_id.
Backend tests	Adds tests covering auth gating, 404, happy path, sorting, winning claim flag, superseded handling, missing source defense, empty responses, and role reads.
Frontend types/API	Adds ConflictClaimCandidate, ConflictCandidatesResponse, useConflictCandidates hook, and candidate cache invalidation.
Frontend resolve form	Changes selection state from selectedClaimIdx to selectedClaimId so the UI submits the exact winningClaimId. Superseded candidates are contextual but not selectable.
Verification in patch notes	Patch notes state backend candidate tests and API suite passed, ruff/mypy for touched backend files passed, TypeScript build passed, and one pre-existing ESLint issue remained.

8.1 Why this patch matters

- The base frontend can infer a selected claim by array index, but the displayed evidence claims and conflict.claimIds ordering can diverge.
- The public evidence endpoint intentionally hides claim IDs, so the UI cannot safely submit winning_claim_id without a dedicated authenticated candidate endpoint.
- This is a production correctness issue because it could silently resolve a conflict to the wrong claim.
- Apply this patch before treating the conflict queue as production-safe.

9. Verification status

9.1 Verification performed in this container

Command/check	Result in this workspace
python -m compileall -q src tests alembic	Passed on the extracted backend tree.
python -m ruff check .	Could not run here: ruff is not installed in the container Python environment.
python -m mypy src	Could not run here: mypy is not installed in the container Python environment.
python -m pytest --no-cov -m "not integration" -q	Could not complete here: test collection requires dependencies such as SQLAlchemy, which are not installed in this container Python environment.
npm ci && npm run build for apps/web	Not run here. The uploaded app has package-lock.json, but dependency installation/build verification should be run in the project environment.

9.2 Verification recorded by the repository

- RELEASE.md records a v0.2.0 release verification with compile, ruff, format, strict mypy, unit/API tests, integration tests, migration round-trip, ORM/constraint checks, secret scans, Docker build, Trivy image/filesystem scan, and Gitleaks passing.
- The report should not pretend those checks were re-run in this ChatGPT container. They are release-doc evidence from the uploaded repository, not live verification from this environment.
- Before shipping new changes, re-run the backend and frontend verification in a proper project environment with dependencies installed.

10. Known gaps and unfinished work

Gap	Current state	Why it matters
BFF/session auth	Skeleton exists in atlas-bff-docs-reports.zip; needs DB/session/CSRF/API-key and apps/web wiring.	Browser must never hold X-API-Key; harden existing BFF.
Human identity model	Backend ApiKeyModel has user_id UUID but no human user/password/session schema.	Needed for real browser login, user management, and audit names.
Conflict candidates in base	Applied in atlas-argus-docs-reports.zip; preserve candidates endpoint/schema/UI.	Needed to prevent wrong winning_claim_id submission.
Report export engine	Reports.tsx exists; production immutable PDF generation/storage/download remains incomplete.	Paid deliverable for legal/investigation customers.
Document upload/ingestion UI	Documents.tsx exists; needs production BFF/storage/parser/status wiring.	Users need evidence intake wired into the system.
Unstructured document ingestion adapter	Structured NTSB importer exists; production PDF/docket parser/storage workflow is still missing.	Legal users live in PDFs, dockets, transcripts, and exhibits.
Marketing website	No apps/marketing site exists.	Needed for public positioning, trust pages, and request-access lead capture.
Request-access/contact backend	No public lead-capture flow is present.	Needed before launch/sales.
Frontend tests	No Playwright suite or production-level frontend integration tests found.	Needed to lock login, case load, conflict resolve/reopen, 409 retry, report export, and upload flows.
Search/map/causality UI	Backend layers exist, but rich UI surfaces are not completed in apps/web.	These should follow after the core conflict/evidence/report loop is solid.

11. Recommended next build sequence

Order	Build next	Reason
1	Verify conflict-candidates patch is present in atlas-argus-docs-reports.zip.	Already applied; prevent regression in the paid workflow.
2	Wire and harden services/bff skeleton: production DB, sessions, CSRF, API-key bridge.	Skeleton exists; next work is integration, not scaffolding.
3	Normalize frontend routes through the BFF and remove any direct browser dependency on FastAPI API keys.	Keeps auth, error shaping, and slug/event ID mapping in one place.
4	Harden app shell and role gating.	Analyst/reviewer/admin behavior must be predictable and enforced in UI and backend.
5	Unify EvidencePanel across claims, conflicts, timeline, provenance, and reports.	Evidence visibility is the product differentiator.
6	Finish conflict queue UX, including candidate selection, 409 refresh, history, and disabled states.	This is the main reviewer workflow.

7	Finish existing Reports.tsx: preview/generate/download/version history.	This turns evidence review into a sellable deliverable.
8	Finish existing Documents.tsx: upload, PDF text extraction, parse status, ingest trigger.	This closes the source-intake gap.
9	Build apps/marketing and request-access/contact flows.	Needed for launch and buyer trust.
10	Add Playwright and integration tests.	Locks the critical flows before expanding search/maps/causality UI.

12. AI handoff guidance

12.1 Do this

- Preserve the existing clean architecture. Add vertical slices through domain/application/infrastructure/presentation instead of broad empty scaffolds.
- Use src/atlas as the backend import root. Do not create atlas_argus package paths.
- Keep all API routers under /api/v1 unless intentionally creating a separate BFF/public website surface.
- Use the real role names: analyst, reviewer, admin.
- Keep the evidence chain as the source of truth and projections as derived views.
- Treat tenant-private data as isolated by default. Do not leak it through public/search/map endpoints.
- Add tests alongside every change, using the existing test style and fake unit-of-work patterns where appropriate.
- For frontend work, preserve apps/web and harden it; do not restart the UI from scratch.

12.2 Do not do this

- Do not silently duplicate projected truth into public pages or reports without provenance.
- Do not let browser JavaScript store or send raw X-API-Key credentials.
- Do not infer conflict winners by index or by display order.
- Do not create a second independent timeline, entity, source, or causality engine when Chronos, Orion, Hermes, and causality contexts already exist.
- Do not let natural-language or AI-generated content create public causal claims without reviewed evidence and provenance.
- Do not build search/maps/causality UI before the core login, evidence, conflict, document intake, and report export loop is reliable.

12.3 Minimal validation checklist for the next AI/code pass

```
cd atlas-argus
python -m compileall -q src tests alembic
ruff check .
mypy src
pytest --no-cov -m "not integration"

cd apps/web
npm ci
npm run build
npm run lint
```

End of current-status report.